Practical 1 – Problem sheet

About the practicals

Programming is, first of all, a skill that is learnt by doing. Therefore it is crucially important that you prepare for, and attend, all six scheduled practical sessions. In these, you will be solving programming exercises while receiving advice and feedback from a tutor. Any remaining problems on the practical sheet should be answered as homework, with feedback again provided by the tutor in the practical sessions. If you would like extra help, you can book extra practical session on Moodle (subject to availability).

Exercises are marked according to their difficulty:

- (S) marks *simple* exercises. These normally focus on one new concept of the language in isolation. The answers are typically short, and often closely related to similar problems discussed in the video lessons, with a specific reference given. Some parts of the answer may already be contained in the code template, so that you only need to fill in the gaps.
- (I) marks *intermediate* exercises. These are still relying on concepts discussed in the video lessons, but you may need to apply them in a different context, or combine several techniques. Examples from the lessons may be helpful, but you will need to identify these on your own. Little, if any, partial code will be provided for you.
- (A) marks *advanced* exercises. These problems are more complex to solve, they may need independent reading on the problem, may require some mathematical insight, and sometimes the use of programming techniques that go beyond what was discussed in the lessons.

If you are a beginner, best start with the (S) questions until you feel confident. In order to do well in the assignments, you should solve *at least* the (I) questions.

About the materials

Apart from this problem sheet, you will find some Java code on Moodle which you need for completing the exercises. Click on that entry on Moodle to download the code as a ZIP file. Unpack this ZIP file to your M: drive.* Start the development environment (BlueJ) at `Start | All programs | Programming tools | BlueJ`. Choose `File | Open project`, select the folder `problemcode_p1`, and click `Open`.

The BlueJ screen will now show several orange-ish boxes. Each of these contains sample code or "skeleton code", that is, an unfinished bit of Java code that is left for you to complete. Double-click on any of the orange boxes to see this code.

You will also see some green boxes "behind" the orange boxes. These represent automated test code *(unit tests)* that is supplied for you to check quickly whether your program works. More on that below. You may double-click on the green boxes to see the underlying Java code – indeed, please do so if you're interested –, but you don't need to understand it at this point.

* If you use Chrome on the classroom PCs, after clicking on the file in Moodle, select a location on your M drive to save the ZIP file to. Then double-click on the little box on the bottom-left, click "Extract", then "Extract all", then the "Extract" button. This will produce a folder `problemcode_p1` .

Exercise 1 (S)

This is a very simple example that will show to you how to use the development environment (BlueJ) and how to run automated tests on your code. The task is:

> Write a Java program that, given an integer n, computes $n^2 + 1$.

A code template is given (double-click on `GettingStarted` to see it). Please proceed as follows.

1) Complete the code template with your code (i.e., add the part that computes n^2+1). Refer to the code from Lesson 1 for a similar example.

2) Click "Compile". If the compiler detects a syntax error in your program, it will output an error message, and mark the place in the code where the error has occurred. In this case, fix the problem and click "Compile" again. (Ask if you're stuck.) Go to the next point only when compilation has succeeded (message: "Class compiled - no syntax errors").

3) Close the editor window, going back to the overview screen with the orange and green boxes.

4) Right-click on the orange box `GettingStarted` and select `int squarePlusOne(int n)` from the pop-up menu. Input a value for n (say, start with 0) and click "OK". Verify that the program indeed returns $n^2 + 1$; if it does not, return to step 2) and correct the program. Repeat this for a few different values.

5) Finally, test your code with the automated unit test: Right-click on the green box `GettingStartedTest`, and select "Compile". Right-click again, and select "Test All". A pop-up window will appear, either with a green or a red bar in the middle (test passed or not passed, respectively). If the test was not passed, click on the entry in the upper list to see more details about the problem.

The unit test is a piece of Java code that automatically runs *your* code with several values of test data (different values of n), and compares the output with expected results. Of course, if you wrote the entire program from scratch, this test would not be available (or you would first need to create it); it is provided in this exercise for convenience, in order to give you some rapid feedback.

Exercise 2 (S)

The purpose of this exercise is to introduce you to testing and debugging techniques. You have been provided with some short Java programs, which are however not working as expected. Your task is to fix these mistakes.

a) Compile-time errors We aim at a program that does the following:

> Given a vector $(x, y, z) \in \mathbb{R}^3$ with integer components, it computes the norm-squared of this vector, i.e., it computes $x^2 + y^2 + z^2$.

A program that claims to do this is already given (see the orange box `BugsA`). However, this piece of code contains several mistakes. It has *compile-time errors*, that is, formal inconsistencies in the program that show up when the Java compiler tries to interpret the code. Please proceed as follows.

1) Double-click on the orange box `BugsA` to open the code editor. Take a moment to read through the code in order to understand how it is supposed to work. (Don't change anything yet.)

2) Press `Compile`. You will see that the code does not compile: an error message is being displayed. Read the error message, and modify the program as needed to fix the error.

3) Repeat 2) until no further errors are reported.

4) Once compilation succeeds, go to the main screen of BlueJ, right-click on `BugsA`, and select `int normSquared(int x, int y, int z)`. Enter an example for the vector (x, y, z) and verify whether the output is the desired one.

5) Also run the unit test `BugsATest` for a consistency check. Right-click on the green box `BugsATest`, select "Compile", then "Test All".

b) Run-time errors As another example, we aim at a program that does the following:

> Given two vectors (a, b, c) and $(x, y, z) \in \mathbb{R}^3$ with integer components, it computes the scalar product of these vectors, i.e., it computes $ax + by + cz$.

Again, a program is given (see the orange box `BugsB`). This time, it compiles correctly, i.e., it is written in correct Java syntax. However, when the program actually runs, it produces incorrect results; it exhibits *run-time errors*. Please proceed as follows to fix the mistakes.

1) Double-click on the orange box `BugsB`. Take a moment to read through the code in order to understand how it is supposed to work; don't change anything yet.

2) Press `Compile`. This time, no error message should appear.

3) Go to the main screen of BlueJ, right-click on `BugsB`, and select `int scalarProduct(int a, int b, int c, int x, int y, int z)`. Enter an example for the vectors (a, b, c) and (x, y, z) and verify whether the output is the desired one. You will likely find that this is not the case!

4) Use the debugger to isolate the problem. Go to the code editor again, and position the cursor in the line starting with: `sum = x...` Select `Tools | Set/clear breakpoint` from the menu. A little stop sign will now appear in the left-hand margin next to the current line.

5) Go to the main screen, and run `int scalarProduct(int a ...)` once more. Now the execution will stop at the line mark above, and a little pop-up window (the debugger window) will appear that displays the current state of the variables of the program. Arrange your screen so that you see the debugger and the editor at the same time. Click `Step` in the debugger window, and watch how the debugger steps through your code line by line, and how the variables change. Can you spot what is going wrong?

6) When you have identified the error, correct it in the editor window, and go back to step 2).

7) Repeat the above for different input values (a, b, c), (x, y, z), until you are satisfied that the program works correctly.

8) Finally, run the unit test for a final check. (It is useful to clear any breakpoints that you have set before.) Right-click on the green box `BugsBTest`, select "Compile", then "Test All". Does your program pass the test? If not, go back to step 4).

Exercise 3 (I)

Now that you're familiar with the code editor, the compiler and the debugger, here's a slightly more independent programming task for you.

> Create a Java program that, given two vectors $v_1 = (x_1, y_1)$ and $v_2 = (x_2, y_2)$ in $\mathbb{R}^2$, computes the area of the parallelogram spanned by these two vectors.

We assume here that the angle enclosed by v_1 and v_2 is between 0 and π (counter-clockwise). The area of the parallelogram is then given by the third component of the cross product $v_1 \times v_2$.

Again, a piece of skeleton code (`Parallelogram`) and a unit test (`ParallelogramTest`) are given. Start from these to develop your program.

Exercise 4 (I)

Your next task is to create a Java program for time conversion. The input of the program is an integer indicating the number of seconds after midnight. The program should convert this into a piece of text indicating hours, minutes, and seconds, as in the following example. Suppose that the value of time (seconds after midnight) is 7987, then the expected output is:

```
It is 2 hours, 13 minutes and 7 seconds since midnight.
```

For simplicity, leave the "hours", "minutes" and "seconds" as plurals even if the preceding number is 1.

Hint: Use integer arithmetic with / (division, discarding remainder) and % (division remainder).

Again, skeleton code (`Time`) and unit test (`TimeTest`) are given. Note that for the unit test to succeed, you need to keep *exactly* to the text format indicated above (including all commas, spaces, etc.).

Exercise 5

Write a program that evaluates the following functions of a real variable x.

(S) $f_1(x) = 2x - 4$,

(S) $f_2(x) = x^3 + 3x - 7$,

(S) $f_3(x) = \cos x + \tan x$,

(S) $f_4(x) = e^{2x} + 5.7$,

(I) $f_5(x) = 7\sqrt{x^4 + \pi}$,

(I) $f_6(x) = y^3 + 2y^4 + \sin(\pi y)$ where $y = \sqrt{x^2 + 1}$,

(A) $f_7(x) = \dfrac{\log(\cos x + 2)}{x^2 + 1} + \sqrt[5]{|x|}$,

(A) $f_8(x) = e^{ax}$ where a is the largest integer such that $x^2 + 6 \geq 2a$,

(A) $f_9(x) = \mathrm{Re}\, \dfrac{4e^{-x} - 5 - 2i}{3 + ix}$.

The example code from Lesson 4, class `WorkingWithFloats`, function `myFunction`, may be useful to get you started.

Code templates are given (see `SomeFunctions`). Note that the associated unit test validates all of the above functions; however, you will see these tests separately in the interface, so you can first complete programming for one function, then test it, then continue with the next one.

For an overview of the built-in mathematical functions in Java, see the documentation page or click `Help | Java Class Libraries` in BlueJ (then navigate to package `java.lang`, class `Math`).

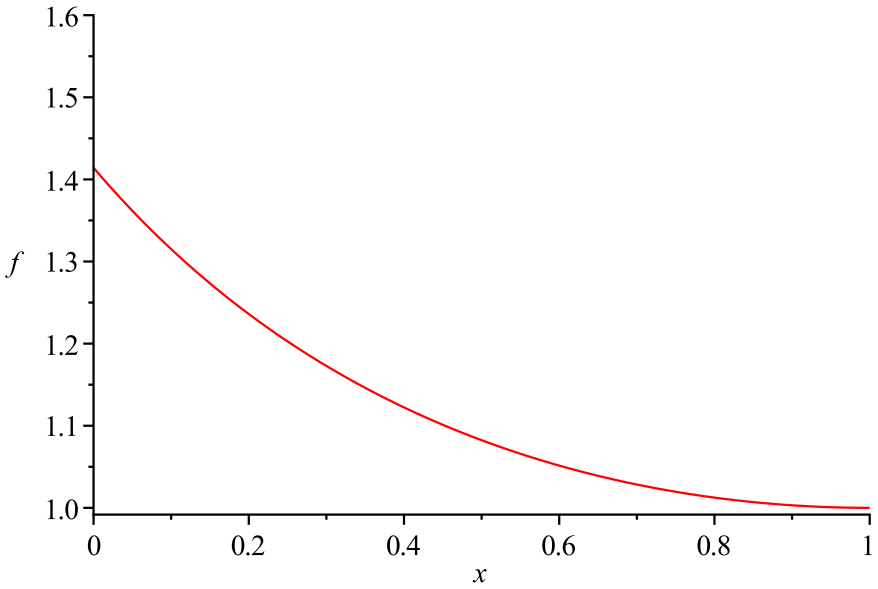
Exercise 6 (A)

We consider the function

$$f(x) = \frac{1 - \tan\frac{(1-x)\pi}{4}}{\cos\frac{(1-x)\pi}{4} - \sin\frac{(1-x)\pi}{4}}, \quad x \in (0,1),$$

see the plot below. Write a program that evaluates this function, using double floating point precision. Test the output very near[†] to $x = 0$ and $x = 1$, and compare it to what is expected from the graph below. Do you notice any problems in accuracy? If so, why do these arise? Can you think of a way to avoid the problem by altering the computation (i.e., by rewriting the function in a suitable way) so that the result becomes more accurate?

Note: There is a code template (`TrigFunctions`), but no unit test for this exercise. Use the graph to check whether your results are plausible.



[†]As a preliminary question: What does *very near* mean, given the floating point precision we use?